

ICM Fatturazioni

Esportazione CSV delle griglie

Progettazione e implementazione della funzione "Esporta CSV"

Versione
1.0

Data
09/07/2026

Stato
Rilasciato

Classificazione
Interno

A CURA DI

Vis Logica

Automazioni e software per PMI
vislogica.com

ANAGRAFICA DOCUMENTO

1 Informazioni sul documento

Titolo	Esportazione CSV delle griglie
Progetto / prodotto	ICM Fatturazioni
A cura di	Vis Logica
Autore	Luca Schiavon
Versione	1.0
Data	09/07/2026
Stato	Rilasciato
Classificazione	Interno

Storico delle revisioni

Versione	Data	Autore	Descrizione delle modifiche
1.0	09/07/2026	Luca Schiavon	Prima stesura: descrizione della funzione “Esporta CSV” delle griglie.

Proprietà

Il presente documento è di proprietà di Vis Logica. Ne è vietata la riproduzione o distribuzione non autorizzata.

SOMMARIO

2 Indice

1 — Informazioni sul documento	2
2 — Indice	3
3 — Introduzione e obiettivi	4
4 — Architettura della soluzione	4
5 — Il generatore CSV (CsvExport)	5
6 — Il download lato client (icmDownloadFile)	6
7 — Integrazione nelle pagine	7
8 — Comportamento e garanzie	9
9 — Sicurezza e conformità	10
10 — Estendere l'export a una nuova griglia	10

PREMESSA

3 Introduzione e obiettivi

Questo documento descrive come è stata realizzata, nell'applicativo **ICM Fatturazioni** (Blazor Server, .NET 10, MudBlazor), la funzione di **esportazione in formato CSV** dei dati mostrati nelle griglie. Ogni pagina interessata espone un pulsante **“Esporta CSV”** che produce un file scaricabile e apribile direttamente in Microsoft Excel.

Gli obiettivi che hanno guidato l'implementazione sono:

- **Fedeltà a ciò che l'utente vede.** L'export riproduce la *vista filtrata corrente* e le *sole colonne visibili* nella griglia.
- **Completezza.** Vengono esportati *tutti* i record del risultato del filtro, non solo la porzione visibile a schermo.
- **Compatibilità con Excel in locale italiano** (separatore, accenti, numeri).
- **Sicurezza.** Nessun dato sensibile e mitigazione delle formule maligne (CSV injection).
- **Riuso e coerenza.** Una sola infrastruttura condivisa, replicata con lo stesso schema su tutte le griglie.

La funzione è disponibile su sei maschere: **Anagrafiche**, **Banche di appoggio**, **Cantieri**, **Stampe fatture**, **Stampa scadenze** (griglia di anteprima) e **Consultazione verbali**.

COME È ORGANIZZATA

4 Architettura della soluzione

La funzione poggia su due componenti riutilizzabili già presenti nell'applicazione (nati per l'export delle pagine di *Audit* e *Log*) più l'integrazione, pagina per pagina, nel componente Blazor della griglia. La responsabilità è nettamente separata:

Componente	Tipo	Responsabilità
Export/CsvExport.cs	Classe statica C#	Trasforma intestazioni + righe in byte CSV (quoting, BOM, anti-injection).
wwwroot/js/download.js	Funzione JavaScript globale	Riceve i byte (in base64) e forza il download del file sul client.
Componente .razor della pagina	UI + orchestrazione	Pulsante "Esporta CSV" e metodo EsportaCsvAsync che mappa i dati e invoca i due componenti.

Flusso di esecuzione

Alla pressione del pulsante, la sequenza è la seguente:

1. Il metodo EsportaCsvAsync della pagina legge la **collezione già filtrata** (la stessa che alimenta la griglia, es. AnagraficheFiltrate).
2. Costruisce le **intestazioni** e converte ogni elemento in una **riga di stringhe**, replicando le colonne visibili.
3. Chiama CsvExport.Genera(...) che restituisce i byte[] del file.
4. I byte vengono codificati in **base64** e passati alla funzione JavaScript icmDownloadFile via interoperabilità (IJSRuntime).
5. Il browser materializza il file e avvia il download; un messaggio (snackbar) conferma l'esito.

Perché il passaggio via JavaScript

In Blazor Server il codice C# gira sul server e non ha accesso al disco del client. Il file va quindi trasferito al browser (come base64) e ricostruito lì in un Blob , che viene scaricato con un link temporaneo. È l'unico modo per "salvare un file" lato utente.

Nota di progetto

Tutte le griglie coinvolte sono *client-side*: l'intero elenco è già in memoria e la virtualizzazione di MudBlazor agisce solo sul rendering, non sui dati. Per questo l'export dell'"intero risultato" non ha richiesto alcuna nuova query o metodo di repository/manager.

5 Il generatore CSV (`CsvExport`)

La classe statica `CsvExport` è l'unico punto in cui si costruisce il testo CSV. Espone un solo metodo pubblico, `Genera`, che riceve le intestazioni e una sequenza di righe (ciascuna già convertita in cella-stringa) e restituisce i byte del file.

```
public static byte[] Genera(IReadOnlyList<string> intestazioni,
                           IEnumerable<IReadOnlyList<string?>> righe)
{
    var sb = new StringBuilder();
    sb.Append(string.Join(Separatore, intestazioni.Select(Escape)));
    sb.Append(FineRiga);

    foreach (var riga in righe)
    {
        sb.Append(string.Join(Separatore, riga.Select(Escape)));
        sb.Append(FineRiga);
    }

    var preambolo = Encoding.UTF8.GetPreamble(); // BOM
    var corpo     = Encoding.UTF8.GetBytes(sb.ToString());
    // ... concatena BOM + corpo e restituisce il buffer
}
```

Scelte di formato (compatibilità Excel IT)

- **Separatore “ ; ” (punto e virgola).** È quello atteso da Excel con impostazioni italiane, dove la virgola è il separatore decimale.
- **Codifica UTF-8 con BOM.** Il preambolo (*Byte Order Mark*) fa sì che Excel riconosca il file come UTF-8: senza di esso interpreterebbe il testo come ANSI e romperebbe gli accenti (à, è, ò...).
- **Fine riga CRLF (`\r\n`),** come da standard RFC 4180.
- **Importi in formato italiano N2** (es. `1.234,56`), così Excel li interpreta come numeri.

Quoting e protezione anti-formula

Ogni cella passa dalla funzione interna `Escape`, che applica il quoting RFC 4180 e neutralizza le formule (protezione da *CSV injection*):

```
private static string Escape(string? valore)
{
    var v = valore ?? string.Empty;

    // Anti CSV-injection: neutralizza i caratteri che Excel tratta da formula.
    if (v.Length > 0 && v[0] is '=' or '+' or '-' or '@')
        v = "'" + v;

    // Quoting se la cella contiene separatore, virgolette o a-capo.
    if (v.Contains(Separatore) || v.Contains('"') || v.Contains('\n') || v.Contains('\r'))
        v = "\"" + v.Replace("\"", "\\\"") + "\"";

    return v;
}
```

In pratica: una cella che inizia con `=`, `+`, `-` o `@` viene fatta precedere da un apostrofo, così Excel non la esegue come formula; le celle che contengono il separatore, virgolette o ritorni a capo vengono racchiuse tra virgolette (con raddoppio delle virgolette interne).

IL SALVATAGGIO LATO CLIENT

6 Il download lato client (`icmDownloadFile`)

La funzione JavaScript globale `icmDownloadFile` (in `wwwroot/js/download.js`) riceve il nome del file, il contenuto in base64 e il *content type*, ricostruisce i byte in un `Blob` e ne forza il download tramite un elemento `a` temporaneo:

```
window.icmDownloadFile = (fileName, base64, contentType) => {  
  const bytes = Uint8Array.from(atob(base64), c => c.charCodeAt(0));  
  const blob = new Blob([bytes], { type: contentType || 'application/octet-stream' });  
  const url = URL.createObjectURL(blob);  
  const a = document.createElement('a');  
  a.href = url;  
  a.download = fileName;  
  document.body.appendChild(a);  
  a.click();  
  document.body.removeChild(a);  
  URL.revokeObjectURL(url);  
};
```

La funzione è **globale e condivisa**: è la stessa utilizzata dalle pagine amministrative di Audit e Log, quindi disponibile in tutta l'applicazione senza duplicazioni.

7 Integrazione nelle pagine

Su ciascuna griglia l'integrazione segue sempre lo stesso schema, molto contenuto (nell'ordine di poche decine di righe per pagina), **senza** alcuna modifica a database, manager o repository:

- direttive `@using ICMFatturazioni.Web.Export` e `@inject IJSRuntime JS` (quest'ultima solo dove non già presente);
- un pulsante **“Esporta CSV”** nella barra strumenti, disabilitato quando non ci sono righe o mentre l'export è in corso;
- un metodo `EsportaCsvAsync` che mappa la collezione filtrata e invoca i due componenti.

Il pulsante

```
<MudButton Variant="Variant.Outlined" Color="Color.Default"
    StartIcon="@Icons.Material.Filled.Download" Ripple="false"
    Disabled="@(_esportazioneInCorso || !AnagraficheFiltrate.Any())"
    OnClick="EsportaCsvAsync">
    Esporta CSV
</MudButton>
```

Il metodo (esempio: Anagrafiche)

```
private async Task EsportaCsvAsync()
{
    var dati = AnagraficheFiltrate.ToList();
    if (dati.Count == 0)
    {
        Snackbar.Add("Nessuna anagrafica da esportare.", Severity.Info);
        return;
    }

    _esportazioneInCorso = true;
    try
    {
        var intestazioni = new[]
        {
            "Ragione sociale", "Tipo", "P.IVA / CF", "Città", "Prov.", "Paese",
            "Email", "Telefono", "Pagamento", "Banca", "Cod. IVA",
        };
        var righe = dati.Select(a => (IEnumerable<string?>)new string[]
        {
            a.RagioneSociale, a.TipoAnagrafica.Descrizione(), a.PIVA, a.City,
            a.Provincia, a.SiglaPaese, a.Email, a.Telefono,
            RisolviPagamento(a.IdPag), RisolviBanca(a.IdBancaAppoggio), RisolviIva(a.IdCodiciIVA),
        });

        var bytes = CsvExport.Genera(intestazioni, righe);
        var nomeFile = $"Anagrafiche_{DateTime.Now:yyyyMMdd_HH:mm}.csv";
        await JS.InvokeVoidAsync("icmDownloadFile", nomeFile,
            Convert.ToBase64String(bytes), "text/csv");
        Snackbar.Add($"Esportate {dati.Count} anagrafiche.", Severity.Success);
    }
    catch (Exception ex)
    {
        await LogManager.LogErrorAsync(ex, "Esportazione CSV anagrafiche fallita.",
            "Anagrafiche.EsportaCsvAsync");
        Snackbar.Add("Errore durante l'esportazione CSV.", Severity.Error);
    }
    finally { _esportazioneInCorso = false; }
}
```

Colonne risolte

Dove la griglia mostra un valore "risolto" da una chiave (es. la descrizione del pagamento o della banca al posto dell'identificativo), l'export riusa le stesse funzioni della UI (`RisolviPagamento` , `RisolviBanca` , ...): il CSV riporta quindi esattamente il testo visibile a video.

Griglie coperte e colonne esportate

Pagina	Colonne esportate
Anagrafiche	Ragione sociale, Tipo, P.IVA/CF, Città, Prov., Paese, Email, Telefono, Pagamento, Banca, Cod. IVA
Banche di appoggio	Intestatario, Banca, ABI, Agenzia, CAB, IBAN
Cantieri	Ubicazione, Attività, Tipologia, Importo appalto
Stampe fatture	Numero, Data, Cliente, Tipo, Attività
Stampa scadenze (anteprima)	Scadenza, T, Cliente, Attività, Dettaglio, Importo, Evasa
Consultazione verbali	Cliente, Attività, Cantiere, Data, Verbale, Stato, CSE, Impresa, Prescr.

Il nome del file segue lo schema `<Entità>_aaaaMMgg_hhmm.csv` (es. `Anagrafiche_20260709_1742.csv`).

COSA GARANTISCE

8 Comportamento e garanzie

- **Vista filtrata.** L'export parte sempre dalla stessa collezione che alimenta la griglia: filtri di testo, tipologia, anno e selezioni sono già applicati.
- **Sole colonne visibili.** Le intestazioni e le celle corrispondono alle colonne a video; la colonna delle azioni (icone) è esclusa.
- **Tutti i record.** Viene esportato l'intero risultato del filtro, non solo la parte renderizzata dalla virtualizzazione.
- **Stato del pulsante.** È disabilitato quando non ci sono righe e durante l'elaborazione, per evitare doppie esecuzioni.
- **Esito comunicato.** Un messaggio conferma il numero di righe esportate o segnala un eventuale errore.
- **Robustezza.** Eventuali eccezioni vengono registrate nel log applicativo (`ILogManager.LogErrorAsync`) senza interrompere l'esperienza d'uso.

9 Sicurezza e conformità

- **Nessun segreto nei file.** Vengono esportate solo le colonne mostrate; non sono inclusi identificativi tecnici, hash o dati riservati non pertinenti.
- **Anti CSV-injection.** Le celle che potrebbero essere interpretate come formula da Excel/Calc vengono neutralizzate a monte (vedi capitolo 5), in linea con le regole di sicurezza del progetto.
- **Accesso protetto.** Le pagine che espongono l'export sono raggiungibili solo da utenti autenticati, secondo le policy di autorizzazione dell'applicazione.

10 Estendere l'export a una nuova griglia

Aggiungere "Esporta CSV" a una nuova pagina richiede pochi passi, senza toccare i livelli dati:

1. Aggiungere `@using ICMFatturazioni.Web.Export` e, se assente, `@inject IJSRuntime JS`.
2. Inserire il pulsante nella barra strumenti, con `Disabled` legato alla presenza di righe.
3. Scrivere `EsportaCsvAsync`: definire le intestazioni, mappare la collezione filtrata in righe di stringhe (riusando le stesse funzioni della griglia per i valori "risolti"), quindi chiamare `CsvExport.Genera` e `icmDownloadFile`.

In sintesi

Tutta la complessità (formattazione compatibile con Excel, quoting, BOM, protezione anti-formula e download nel browser) è incapsulata in `CsvExport` e `icmDownloadFile`. La singola pagina deve solo dichiarare *quali colonne* e *quali dati* esportare.